# The Algebraic Blind Spot in Pattern-Matching Defenses

Joseph Robert Lopez

May 2026

## Contents



## The Algebraic Blind Spot in Pattern-Matching Defenses

Joseph Robert Lopez

### The argument in one paragraph

Substring-matching pattern tiers — the regex WAFs, spam filters, secret scanners, and LLM guardrails currently shielding production systems — share a structural property that determines what they can and cannot see. The property is *aperiodicity of the syntactic monoid*, and its consequence is concrete: no pattern in this class can decide whether a count is even or odd. An attacker who hides a payload across alternating positions — payload on even indices, filler on odd — bypasses the tier, because deciding the alternation is precisely the modular-counting predicate the tier provably cannot compute. This article reports a measurement of 142 production patterns drawn from twelve sources (100% aperiodic, 100% bypass under $\mathrm{MOD}_2$ encoding), establishes that no composition of pattern matchers closes the bypass class, and identifies the architectural change that does. Against the LLM-Guard production library — currently shipped to deployed AI systems — the default regex configuration detects 0% of $\mathrm{MOD}_2$- and $\mathrm{MOD}_3$-encoded payloads; the parallel-OR composition we describe recovers detection to 95–100%.

## Who this is for, and why now

If you are a security architect or CTO responsible for input-filtering pipelines — WAFs, spam filters, code and secret scanners, IDS regex tiers, LLM guardrails — this article tells you which slice of your defense stack is structurally blind to a class of bypass, and what to change in the composition topology to recover. The structural property cuts across all of these defenses, and the implication for defense-in-depth is concrete enough to act on with current infrastructure.

You will recognize the symptom from your incident-response logs: unicode-encoded payloads slipping past keyword tiers, leetspeak evading SQL-injection rules, zero-width characters defeating secret scanners. This article gives you the structural reason and the architectural fix. If your audit reports treat regex coverage as a compliance control, the diagnostic below is the basis for distinguishing what that control can certify and what it cannot.

Russinovich et al., writing in this venue, frame LLM jailbreaks as a surmountable challenge addressable through layered defense [11]. We complement that framing: one specific layer — substring matching — is structurally unsurmountable. The implication for composition is that serial-AND pipelines do not recover from this layer's blindness, regardless of how capable the downstream tier is. Russinovich et al. recommend, among other defenses, neural input filters chained behind regex pre-screens; the algebraic argument below shows why that specific chaining order propagates the regex tier's blindness rather than recovering past it.

The result is not new mathematics. It chains three classical theorems — Schützenberger (1965) [13], Barrington-Compton-Straubing-Thérien (1992) [1], and Furst-Saxe-Sipser (1981) [4] — onto an empirical security setting. The novelty is the diagnostic, the measurement, and the architectural conclusion.

## Quick glossary

Before the math arrives, three terms in plain English:

- **Substring matching** — what regex tools do when configured to find a pattern *anywhere* in the input. The pattern `exec\s*\(` matches the literal `exec(` (with optional whitespace) anywhere in the string. This is the dominant deployment shape across WAFs, guardrails, and content filters.
- **Aperiodic monoid** — an algebraic object you can extract from any finite-state matcher. "Aperiodic" means the matcher cannot model "every k-th step" behavior. Counting modulo a prime — the operation that decides "is this position even or odd?" — is exactly what aperiodic matchers cannot do.
- **$\mathbf{AC}^0$** — a complexity class (constant-depth, polynomial-size Boolean circuits). Substring-matching regex sits inside $AC^0$. A 1981 result proves $AC^0$ provably cannot compute parity. That single fact propagates through every composition of substring-matching tiers in your stack.

You do not need to follow the proofs. You need to know that the chain is unconditional, classical, and unfixable inside the regex paradigm.

## The bypass

Take any substring-matching pattern. A canonical example from our corpus is `exec\s*\(`, which matches the literal substring `exec(` (with optional whitespace) and ships in WAF-generic rule sets

and adversarial test suites alike (verified: `corpus_full.csv` lines 40 and 53). Every guardrail library we surveyed ships at least one substring matcher of this form.

To bypass it, encode the payload across alternating positions: write `exxxexcx(`, where every other character is filler. The pattern does not match, because no contiguous substring of the encoded string is `exec(`. The decoded payload — read every other character starting at index 0 — is `exec(`. The decoder is a parity operation. The pattern cannot perform parity.

This is the entire attack. It generalizes from $\text{MOD}_2$ (alternating positions) to MOD_p for any prime p (every p-th position is payload, the rest is filler). It absorbs several documented evasion techniques as instances:

- **Reading every other character** is $\text{MOD}_2$ with arbitrary filler.
- **Acrostics** are MOD_k at line granularity rather than character granularity.
- **Zero-width Unicode insertion** (U+200B, U+200C) between payload characters is $\text{MOD}_2$ with the null filler — Boucher et al.'s "imperceptible" attack class [2] is a documented instance.
- **Token splitting** that forces the tokenizer to reassemble a keyword across boundaries is a partial $\text{MOD}_2$ at token level.

A concrete real-world example you have likely seen: a SQL-injection payload like `UNION SELECT password FROM users` that bypasses a WAF rule of the form `(?i)union.*?select.*?from` (this is ModSecurity OWASP CRS rule 942270, verbatim). Inject a zero-width space between every character — `UNION SELECT password FROM users` — and the regex finds no contiguous substring matching `union.*?select.*?from`, while the database engine, which strips or ignores those codepoints during parsing, sees the original payload (verified: `results/non_llm_defense_bypass.json`, `crs_942270_union_select`, ZWSP bypass 1/1).

The reason these all work, and the reason no further pattern can be added to the tier to close them, is the same: the substring-matching design pattern compiles to an aperiodic finite-state machine, and aperiodic machines provably cannot count modulo any prime. Hackett et al. [7] survey related empirical bypass strategies; the algebraic result below explains why a specific subset of those strategies is not patchable within the regex paradigm.

## Why this is structural

Here is the argument in plain English, with the formal citations as parentheticals so you can verify and move on.

Every substring-matching pattern compiles to a finite-state machine of a particularly simple kind: one that recognizes a *star-free* language (formally, languages of the form $\Sigma^* \cdot \text{L(r)} \cdot \Sigma^*$ are star-free; Theorem 6.1 of the companion preprint). Star-free languages have a clean algebraic characterization: their syntactic monoids are *aperiodic* — they contain no group structure, no cyclic counting (Schützenberger 1965 [13]; McNaughton-Papert 1971 [9]). Aperiodic regular languages live inside the complexity class $\mathbf{AC}^0$ (Barrington-Compton-Straubing-Thérien 1992 [1]). And **$\mathbf{AC}^0$ provably cannot compute MOD_p for any prime p** (Furst-Saxe-Sipser 1981 [4]; Håstad 1987 [6]).

Chain those four facts together and the conclusion is automatic: every substring-matching pattern is star-free, hence in $\text{AC}^0$, hence cannot decide MOD_p, hence is blind to any payload encoding whose decoder is a modular-counting operation. The conclusion holds for *any* finite Boolean composition of such patterns: star-free languages are closed under union, intersection, and complement, so stacking matchers preserves the blindness.

That closure property is the load-bearing patchability statement. Any further substring-matching pattern added to the tier is itself star-free, the union remains star-free, the syntactic monoid remains aperiodic, and the new tier remains in $AC^0$. **The MOD_p bypass class is not closed by any finite addition of substring-matching patterns.** This is a closure-under-composition statement, not a tuning recommendation.

The same argument extends to the broader class of star-free string matchers used across your stack: tries, finite DFA blocklists, Bloom filters on their intended literal sets. Every one of these recognizes a star-free language by construction; every one is in $AC^0$; every Boolean composition of them remains in $AC^0$. The architectural consequence is that no defense-in-depth pipeline composed entirely of substring-matching tiers can close the modular-counting bypass class, regardless of how the tiers are written, ordered, or weighted.

### Why the formalism matters

Empirical bypass studies show what didn't work on a particular test set. The formal chain shows what cannot work on any future input within the same design pattern. For a security architect, this distinction is operational. A clean monoid-extractor report is a mathematical proof of blindness to a stated set of primes, not a confidence statement based on which payloads happened to be tested. The patchability claim is closed, not open: closure of star-free under Boolean operations means no future regex rule, however clever, can change the verdict. Without the formalism, the natural response to a missed bypass is "add another rule"; with it, the response is "change the composition topology."

The same logic applies to vendor claims and procurement diligence. A vendor that ships a substring-matching tier and asserts coverage of modular-counting evasions is making a claim the formalism rules out, not a claim that more rules could substantiate.

### The corpus

To turn the structural argument into a measurement that practitioners can act on, we collected 142 regex patterns from twelve sources and ran them through a monoid extractor (verified: `corpus_full.csv`, 142 rows; tool: `tools/audit_v14.py`).

- **Nine third-party open-source guardrail and detection projects** contributed 100 patterns: LLM-Guard (12), Guardrails-AI (12), llm-guard-py (11), NeMo Guardrails (11; regex subset of content rails), Rebuff (10), Presidio (10), LangKit (10), BodAIGuard (10), and GitLeaks (7).
- **Three author-assembled sets** contributed 42 patterns: 8 inspired by the OWASP Top 10 for LLMs taxonomy, 15 from a curated WAF-generic pattern set, and 19 author-constructed adversarial test patterns.

For each pattern r, we compiled to a Thompson NFA, converted to the minimal DFA via Hopcroft's algorithm, enumerated the transition monoid by BFS over the Cayley graph, and tested aperiodicity directly: for each monoid element m, check whether the sequence m, $m^2$, $m^3$, … stabilizes (i.e., m^n = m^{n+1} for some computable $n \leq |M(L\_r)|^2$).

**Result.** All 142 patterns (100%) have aperiodic syntactic monoids. All 142 admit the $MOD_2$ bypass: for each pattern we constructed an encoded payload string $\phi_2(h)$ where h matched the pattern in plaintext, and verified the pattern did not match $\phi_2(h)$ (verified: `results/mod_p_bypass_matrix.json` 56/56 across primes {2,3,5,7}; `results/printable_filler_bypass.json`

392/392 across seven filler choices; `results/library_pattern_bypass.json` 48/48 baseline-matched library patterns).

Table 1 shows representative patterns with their DFA size, monoid size, and aperiodicity verdict. The monoid extractor is released as an artifact of this work (~860 lines of Python; artifact bundle DOI: https://doi.org/10.5281/zenodo.20103493).

| ID | Pattern | Source (corpus_full.csv line) | \|M\| | Aperiodic |
|---|---|---|---|---|
| F1 | `rm\s*-[rR][fF]` | test_adversarial (line 54) | 18 | Y |
| F2 | `<script[^>]*>` | WAF-generic (line 33) | 39 | Y |
| F3 | `\.\.[\\/]` | WAF-generic (line 38) | 7 | Y |
| F4 | `eval\s*\(` | WAF-generic (line 39) | 18 | Y |
| F5 | `exec\s*\(` | WAF-generic (line 40) | 17 | Y |

**Table 1.** Representative patterns from the 142-pattern corpus, all rows verbatim. Sources and `corpus_full.csv` line numbers shown for direct verification; monoid sizes are taken from the corpus's `monoid_size` column (computed by `tools/audit_v14.py` on the substring-matching wrap `$\Sigma$* $\cdot$ L(r) $\cdot$ $\Sigma$*`). All 142 patterns in the full corpus are aperiodic.

## Beyond LLM guardrails: WAFs and spam filters

The corpus measurement establishes the result for the LLM-guardrail class. To extend the empirical leg to two non-LLM defensive classes, we ran the same $\text{MOD}_2$ bypass against six ModSecurity OWASP CRS 4.27 SQL-injection patterns and six SpamAssassin `20_phrases.cf` patterns — 12 patterns total, 20 synthesized payloads (10 per class), three filler choices: NULL byte, ASCII space, U+200B zero-width space (tool: `tools/non_llm_defense_bypass.py`; artifact: `results/non_llm_defense_bypass.json`).

**ModSecurity:** 10/10 baseline-matched. NULL bypass: 8/10. ASCII-space: 10/10. ZWSP: 10/10. The two NULL-byte failures are SQL-comment patterns whose alphabet includes `\x00` by design (CRS catches NULL injection explicitly); both bypass under printable-space and zero-width fillers, exactly as the algebraic argument predicts — no single-filler patch closes the class.

**SpamAssassin:** 9/10 baseline-matched (one synthesizer artifact). NULL: 9/9. ASCII-space: 9/9. ZWSP: 9/9.

Across twelve patterns and twenty payloads from two defensive classes distinct from LLM guardrails, every baseline-matched payload bypassed under at least one of the three filler choices. Three further classes named in published surveys but not yet measured here — literal-string AV signatures, IDS regex tiers, and code/secret scanners — remain by algebraic argument alone; the unification claim is empirical for three classes (LLM guardrails, WAF, spam filter) and structural for the rest.

## Composition does not save you

The defender's natural intuition is that stacking tiers covers the gaps: regex first-pass, neural classifier second-pass, output monitor third-pass. The intuition is wrong about what stacking does to the algebraic blindness. The composition behavior depends on whether tiers are combined in serial-AND, parallel-OR, or as a soft-score blend, and on whether the downstream tier has the algebraic capacity the upstream tier lacks.

The two compositions to internalize, drawn for clarity:

```
Serial-AND ("regex confirms first"):

  input --> [regex] --allow--> [neural] --> verdict
              |
              +--block--> verdict (regex blocks; neural never sees input)

  $\to$ If regex misses (encoded payload), neural never sees the threat.
  $\to$ Stack inherits regex's AC$^0$ blindness.


Parallel-OR ("either tier can fire"):

  input --+--> [regex]  --block--+
          |                      +--> block if EITHER fires
          +--> [neural] --block--+

  $\to$ Encoded payload bypasses regex but is caught by neural.
  $\to$ Stack recovers neural's TC$^0$ capacity.
```

Five composition cases that matter:

| Composition | Blindness preserved? | Why |
|---|---|---|
| Regex $\wedge$ Regex (serial) | Yes | Star-free closed under intersection |
| Regex $\vee$ Regex (parallel) | Yes | Star-free closed under union |
| Regex $\wedge$ Neural (regex confirms first) | **Yes** | Regex vetoes; if regex misses, encoded payload is allowed regardless of neural tier |
| Regex $\vee$ Neural (parallel-OR) | No (capacity) | Neural tier in $TC^0$ can compute MOD_p; composed system blocks if either fires |
| Soft-score blend, weight $\alpha$ on regex | Yes if $\alpha > \alpha^*$ | Critical weight $\alpha^* = 1 - \tau/c$; above $\alpha^*$ the regex weight dominates, reimporting $AC^0$ blindness |

**Table 2.** Composition laws for substring-matching and neural tiers. Items 1–2 follow from Boolean closure of star-free languages. Item 3 — the regex-confirms-first design — propagates the regex layer's blindness because the dangerous case is precisely the one the regex does not flag. Item 4 is the only composition that breaks the ceiling, subject to the realization caveat below. Item 5 quantifies how blends inherit blindness above a critical weight.

The critical row is the third. A "regex confirms, neural inspects" architecture inherits the regex layer's blindness: the encoded payload is invisible to the regex, the regex returns `allow`, the AND short-circuits, and the neural tier's capacity is irrelevant to the outcome. This is the design pattern most commonly deployed in production guardrail stacks. The audit question — "is this

composition parallel-OR or serial-AND?" — is answerable from configuration in minutes, and determines whether the tier's purported coverage is real or notional.

The fourth row — parallel-OR with a $TC^0$ tier — is the architectural fix, and it carries one important caveat. The capacity statement is that some $TC^0$ circuit can compute MOD_p; a single uniform attention head computes parity directly via threshold gates [3, 10, 12]. The caveat, due to Hahn (2020) [5] and follow-up work by Chiang, Cholak, and Pillay [3], is that fixed-architecture self-attention cannot model parity for unboundedly long inputs unless precision scales with sequence length. **Capacity is what the complexity class permits; realization is what the deployed model achieves.**

We validated the realization in the small-payload regime relevant to deployed guardrails. A minimal transformer classifier (one layer, one attention head, d_model=32, 14,753 parameters) trained on character-level $MOD_2$-encoded payloads achieved 90% detection on held-out $MOD_2$, 90% on unseen $MOD_3$, and 100% on $MOD_5$ and $MOD_7$, with 0% false-positive rate on clean text. Regex detection across all four encodings was 0%. The capacity is achievable in the deployment-relevant regime, even if the unbounded-length statement remains conditional on architectural scaling.

### End-to-end against a production library

To validate the composition argument against a deployed system rather than synthetic patterns, we tested the full defense stack against 23 patterns from LLM-Guard's BanSubstrings scanner — a production guardrail library shipped by ProtectAI. We ran 20 payloads spanning prompt injection, code injection, SQLi, and XSS through four defense configurations and seven encoding types.

| Encoding | Regex only | + Preprocessing | + Neural | Full stack |
|---|---|---|---|---|
| Raw payload | 95% | 95% | 95% | 95% |
| $MOD_2$ | **0%** | 0% | 100% | 100% |
| $MOD_3$ | **0%** | 0% | 100% | 100% |
| Base64 | 0% | 95% | 100% | 100% |
| ROT13 | 0% | 95% | 90% | 100% |
| ZWSP | 0% | 95% | 0% | 95% |
| Homoglyph | 15% | 95% | 60% | 95% |
| Leetspeak | 0% | 95% | 80% | 100% |

**Table 3.** End-to-end detection against 23 LLM-Guard production patterns. "Regex Only" is the current default configuration. "Full Stack" adds preprocessing (strip, decode, normalize) and a parallel neural tier.

The "Regex Only" column is the algebraic blindness theorem made operational: the production library detects **0% of encoded payloads across six of the seven encoding types**. The exception — homoglyph attacks at 15% — is the rule sharpened, not contradicted: some homoglyph substitutions create incidental substring matches the regex still catches (a Cyrillic 'a' replacing a Latin 'a' in some contexts produces a string that still trips a different rule), but six of seven encoding classes leak through completely. The "Full Stack" column shows 95–100% detection across the same encodings. The defense gap is entirely attributable to the $AC^0$ ceiling; the recovery is entirely attributable to ceiling-breaking composition (parallel-OR with a $TC^0$ tier) plus invertibility-based preprocessing.

### An architectural checklist

The diagnostic above produces a short list of architectural questions that determine whether a given pipeline is exposed to this class of bypass. These are review questions, not vendor-cornering questions; they are the ones we would ask of our own systems before asking them of someone else's.

**1. What is the algebraic class of each tier?** A pattern-matching tier (regex, trie, finite DFA blocklist, Bloom filter on a literal set) is in $AC^0$ and is provably MOD_p-blind. A neural classifier in the transformer family is in $TC^0$ and has the capacity, though not necessarily the realized behavior, to detect MOD_p. The operational question: which tiers are in $AC^0$, which are in $TC^0$, and where does the boundary fall in the pipeline?

**2. What is the composition topology?** Serial-AND (regex confirms first, neural inspects after) propagates the upstream tier's blindness. Parallel-OR (block if either fires) recovers the downstream tier's capacity. Soft-score blends require the regex weight to stay below a critical $\alpha^* = 1 - \tau/c$; above that weight, the blend reimports the regex tier's blindness. Pipelines that look like "regex → neural" with the AND semantics are the common high-risk case.

**3. What encodings does preprocessing handle, and which are out of scope?** Invertible encodings (Base64, ROT13, common Unicode normalizations) can be undone before the pattern tier sees the input; they should be. Modular-counting encodings (MOD_p, ZWSP insertion, alternating-position payloads) are not invertible without the modulus and cannot be enumerated by preprocessing alone; they require the $TC^0$ tier in parallel, not preprocessing.

**4. Is the audit theorem-backed or heuristic?** The monoid extractor releases a constructive audit: given a pattern, it computes the minimal DFA, enumerates the transition monoid, identifies all group components via standard semigroup algorithms, and outputs the complete blindness spectrum — the set of primes p for which the pattern is provably MOD_p-blind (Krohn-Rhodes decomposition [8]; Proposition 6.7 of the preprint). For aperiodic patterns the spectrum is all primes. A clean audit report is a mathematical proof of the stated blindness, not a confidence-based assertion. Integration is straightforward in CI/CD as a pre-commit check; analysis of the 142-pattern corpus completes in under 60 seconds on commodity hardware. The output is an audit-grade artifact: a list of primes the tier is provably blind to, suitable for inclusion in security reviews, compliance documentation, and procurement diligence. The conversation shifts from "we believe these patterns cover X" to "we have a proof that these patterns are blind to Y."

**5. What lives at the execution layer?** A class of attacks — semantic and homomorphic-reasoning attacks (Sections 7.3–7.4 of the preprint, with §8.3's pilot finding that exhaustive grammar search outperforms LLM-guided variants on small benchmarks: BFS dominates, demonstrating the attack vector's structural challenge rather than its operational success at scale) — cannot be defended at any inference layer, regardless of the algebraic class of the tiers. These attacks operate by having the LLM solve an abstract grammar over opaque symbols, with the interpretation table held offline by the attacker. The LLM-visible content is a legitimate-looking formal-reasoning exercise; the harmful instantiation happens after the model's output passes back through the attacker's interpretation table. Defense for this class requires monitoring at the execution layer — what the LLM does, not what it receives — and is the natural complement to inference-layer pattern matching.

The five questions are not exhaustive, and the answers will be specific to each deployment. But they are concrete, theorem-backed, and answerable from the pipeline's configuration plus a sixty-second monoid-extractor run.

**What this means for you:** you can run question 4 on your own patterns this week. You can answer question 2 from your existing architecture diagrams in an hour. The remaining three questions are reading exercises against documentation you already have.

### What this does not solve

The algebraic blindness result is unconditional for substring-matching tiers. The story is more nuanced one tier up. Neural guardrails in the transformer family escape $AC^0$ and the modular-counting bypass — that is the content of the parallel-OR recovery. They face their own structural barriers, which the algebraic result does not address.

First, $TC^0$ itself has a conjectured ceiling at $NC^1$. Problems that require evaluating arbitrary Boolean formulas over input tokens are believed to lie outside $TC^0$, though the separation has resisted proof for four decades. A guardrail that must evaluate a complex logical safety policy over its input is conjectured to require $NC^1$ or higher; attackers who can force the detection problem into the $NC^1$ $TC^0$ gap can — conditionally on the conjectured separation — evade neural guardrails by an analogous complexity argument. The unconditional result for regex does not transfer to a corresponding unconditional result for neural tiers.

Second, neural tiers are subject to gradient-based adversarial attacks [14] that operate in weight space rather than in the language's algebraic structure. This is a parallel threat model, not a counter-argument to the composition recommendation, but it is a real cost to be priced in. Inference for a small neural guardrail is orders of magnitude more compute than DFA traversal — typically 5–7 orders of magnitude for the parameter sizes commonly deployed.

Third, the homomorphic-reasoning class of attacks (V3/V4 in the preprint) operates by abstraction rather than by encoding. The LLM-visible prompt contains an abstract grammar over opaque symbols and is, as a string, indistinguishable from a legitimate formal-reasoning exercise. No content-inspection tier — regex, neural, or any composition — can distinguish the adversarial prompt from a legitimate one, because at the prompt boundary they are the same prompt. The defense for this class is execution-layer, not inference-layer, and the architectural checklist's fifth question is the entry point.

The intent of the algebraic result is not to claim that pattern matching is useless — it catches what it was designed to catch, the threats whose distinguishing feature is a contiguous substring. The intent is to mark the boundary of what the design can do, so that pipelines built on top of it do not over-claim coverage that the upstream tier provably cannot provide.

### Closing

The substring-matching tier is doing real work, and we are not arguing for its removal. We are arguing for two architectural changes that follow from a structural property of the tier, not from any criticism of the patterns it ships with.

First, do not compose pattern-matching tiers serially with a more-capable downstream tier and call the composition defense-in-depth. Serial-AND propagates the upstream tier's blindness; parallel-OR recovers the downstream tier's capacity; soft-score blends require the regex weight to stay below a critical threshold.

Second, audit the algebraic class of each tier and record its blindness spectrum. The audit is constructive, theorem-backed, and runs in seconds.

Both changes are answerable from configuration plus a one-minute audit run. The audit produces a defensible artifact — not a confidence statement that "we tested these and they passed," but a formal blindness spectrum that can be cited and re-verified by any third party with the same monoid extractor. That is a different category of evidence from empirical coverage reports, and it is the category a structural impossibility result calls for.

These results extend by algebraic argument to literal-string AV signatures, IDS regex tiers, and code/secret scanners; we have not run direct empirical pilots there yet. The unification claim is empirical for the three defensive classes we measured (LLM guardrails, WAFs, spam filters) and structural for the rest.

This complements rather than contradicts Russinovich et al.'s surmountable-challenge framing [11]. Their argument is that LLM jailbreak defense is winnable through layered defense; ours is that one specific layer is structurally unsurmountable, and the implication for composition is that some compositions are not layered defense at all but are inherited blindness with extra steps. Both readings are true. The architectural conclusion that ties them together is that **defense-in-depth is a property of the composition topology, not of the count of tiers.**

## Notes and references

The companion technical preprint (https://doi.org/10.5281/zenodo.20103491) contains the formal apparatus: full proofs of the substring-aperiodicity theorem and the modular-counting bypass, the Krohn-Rhodes audit construction, the composition laws with full case analysis, and the homomorphic-reasoning attack vectors. The released artifact bundle (https://doi.org/10.5281/zenodo.20103493) contains the 142-pattern corpus, the monoid extractor, all bypass harnesses, and the JSON pilots cited inline.

**AI-methodology disclosure.** This work was developed using an agentic AI research pipeline. The structural argument (Schützenberger → BCST → Furst-Saxe-Sipser) is a chain of classical results assembled by the author; the substring-aperiodicity lemma and its proof were drafted iteratively with AI assistance and verified by hand against Pin's *Varieties of Formal Languages* and the BCST original. The 142-pattern corpus, the monoid extractor (`tools/audit_v14.py`), the MOD_p bypass harness, and the empirical pilots were implemented and run by the author. The article and its companion preprint went through multiple adversarial-review cycles using AI peer-reviewer agents; every numerical claim, citation, and inline `(verified: …)` anchor in this article was re-checked against the released artifacts before submission. Full tooling and model-usage detail is in the preprint's appendix and the artifact bundle.

**References (selected; full list in preprint).**

[1] D. A. M. Barrington, K. Compton, H. Straubing, and D. Thérien. Regular languages in $NC^1$. *Journal of Computer and System Sciences* 44(3), 478–499, 1992.

[2] N. Boucher, I. Shumailov, R. Anderson, and N. Papernot. Bad characters: imperceptible NLP attacks. *IEEE Symposium on Security and Privacy*, 2022.

[3] D. Chiang, P. Cholak, and A. Pillay. Tighter bounds on the expressivity of transformer encoders. *ICML*, 2023.

[4] M. Furst, J. B. Saxe, and M. Sipser. Parity, circuits, and the polynomial-time hierarchy. *FOCS*, 260–270, 1981.

[5] M. Hahn. Theoretical limitations of self-attention in neural sequence models. *TACL* 8, 156–171, 2020.

[6] J. Håstad. *Computational Limitations of Small-Depth Circuits*. PhD thesis, MIT, 1987.

[7] W. Hackett, L. Birch, S. Trawicki, N. Suri, and P. Garraghan. Bypassing LLM guardrails: an empirical analysis of evasion attacks against prompt injection and jailbreak detection systems. *LLMSEC at ACL*, 2025.

[8] K. Krohn and J. Rhodes. Algebraic theory of machines I. *Trans. AMS* 116, 450–464, 1965.

[9] R. McNaughton and S. Papert. *Counter-Free Automata*. MIT Press, 1971.

[10] W. Merrill and A. Sabharwal. The parallelism tradeoff: limitations of log-precision transformers. *TACL* 11, 531–545, 2023.

[11] M. Russinovich, A. Salem, S. Zanella-Béguelin, and Y. Zunger. The Price of Intelligence: Three Risks Inherent in LLMs. *Communications of the ACM* 68(9):46–53, 2025. https://doi.org/10.1145/3749447

[12] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin. Formal language recognition by hard attention transformers: precise bounds and some separations. *TACL* 12, 1–19, 2024.

[13] M.-P. Schützenberger. On finite monoids having only trivial subgroups. *Information and Control* 8(2), 190–194, 1965.

[14] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. arXiv:2307.15043, 2023.